Cosmos Rehydration - UCX Services

Rehydration is the process of re-processing the events when we have event sourcing (cosmos or any source) when we want to resend the message/event for some purpose. it's like replay in kafka, but as we have middle-ware event sourcing then instead of going back to previous step to kafka replay we can re-generate the state by collecting all messages/events related to a single or all request and sort them in sequence according to their sent date, create the state out of it and then send the further events as needed.

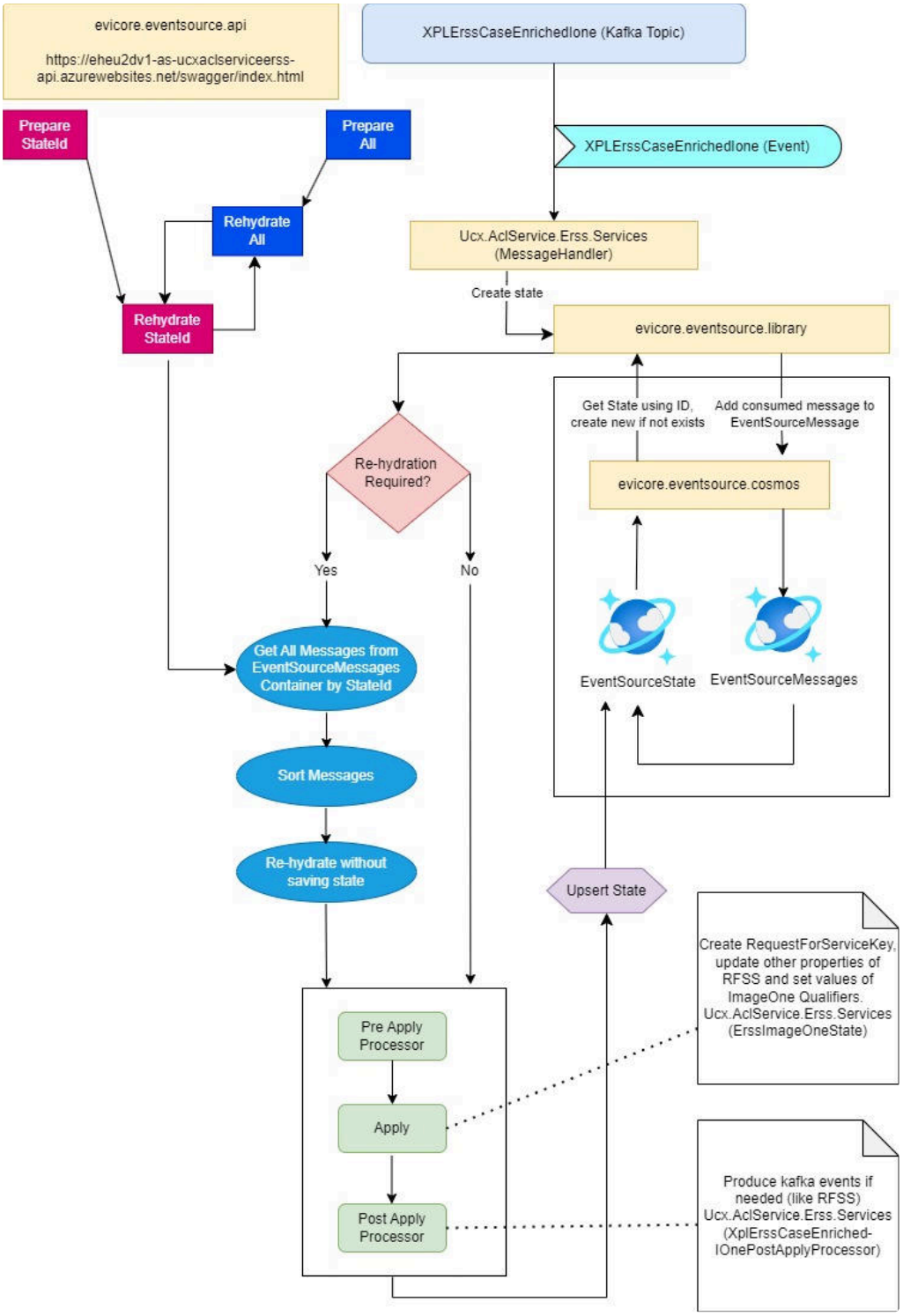
When we need rehydration?
When we want to bump the event version.
When we are correcting any mapping in event fields (if earlier incorrect mapping)
When we are adding new fields in the event, then for historical events when we want to populate those fields.

When we cannot do rehydration and need to go for kafka replay?
Whenever we do not have new fields available in event source messages then we cannot get that data by rehydration. In this case we need to replay kafka topic which will further rehydrate the event source as well.

Pre-requisites:

Prepare/all is the #1 end point we need to hit to start the full rehydration process, before we kicked off the end point we need to adjust the RU's on all containers which are going to hit in this process for any update or insert bulk of records in short time span.

Rehydration Workflow Diagram



ACLService.Erss

Introduction	The ACLService.Erss Service primarily consumes the Kafka topic XPLErssFinal and XPLErssCaseEnrichedIone to produce platform agnostic events for UCX. Also, it acts as an ACL layer for UCX.

Getting Started	There are two applications in this git repository, Ucx.AclService.Erss.Api and Ucx.AclService.Erss.App. There is a `Properties/launchSettings.json` for both projects that have local run/debug configurations.
Event Source	The data for this service is event sourced. It uses evicore.eventsource and evicore.eventsource.cosmos.
Rehydration End Points	Dev: Swagger UI (eheu2dv1-as-ucxaclserviceerss-api.azurewebsites.net) INTG: Swagger UI (eheu2in1-as-ucxaclserviceerss-api.azurewebsites.net) PROD: Swagger UI (ehcuspd1-as-ucxaclserviceerss-api.azurewebsites.net)
Methods	1. Prepare all ◦ This has to complete before #2 should be kicked off. ◦ This is a background task and the endpoint will return an immediate `202 Accepted`. POST request DEV URL- https://eheu2dv1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/prepare/all Content-Type: application/json Settings - {"throttleMs":5,"takePerThrottle":5} POST request INTG URL- https://eheu2in1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/prepare/all Content-Type: application/json Settings - {"throttleMs":5,"takePerThrottle":5} POST request PROD URL- https://ehcuspd1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/prepare/all

	Content-Type: application/json
	Settings - {"throttleMs":5,"takePerThrottle":5}
	
	2. Rehydrate all
	  - This has to start after #1 completes.
	  - This is also a background task and the endpoint will return an immediate `202 Accepted`.
	
	POST request DEV
	URL - https://eheu2dv1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
	Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}
	
	POST request INTG
	URL - https://eheu2in1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
	Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}
	
	POST request PROD
	URL - https://ehcuspd1-as-ucxaclserviceerss-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
	Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}

Decision Service

Introduction	The Decision Service primarily consumes the Kafka topic XPL UpadsReviewCompletedv2 to produce UCX Decision Made messages. It acts as an ACL layer for UCX.

Getting Started	There are two applications in this git repository, Ucx.DecisionService.Api and Ucx.DecisionService.Observer. There is a `Properties/launchSettings.json` for both projects that have local run/debug configurations.
Event Source	The data for this service is event sourced. It uses evicore.eventsource and evicore.eventsource.cosmos.
Rehydration End Points	Dev: Swagger UI (eheu2dv1-as-ucxdecisionservice-api.azurewebsites.net) INTG: Swagger UI (eheu2in1-as-ucxdecisionservice-api.azurewebsites.net) PROD: Swagger UI (ehcuspd1-as-ucxdecisionservice-api.azurewebsites.net)
Methods	1. Prepare all ◦ This has to complete before #2 should be kicked off. ◦ This is a background task and the endpoint will return an immediate `202 Accepted`. POST request DEV URL- https://eheu2dv1-as-ucxdecisionservice-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/prepare/all Content-Type: application/json Settings - {"throttleMs":5,"takePerThrottle":5} POST request INTG URL- https://eheu2in1-as-ucxdecisionservice-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/prepare/all Content-Type: application/json Settings - {"throttleMs":5,"takePerThrottle":5} POST request PROD URL- https://ehcuspd1-as-ucxdecisionservice-api.azurewebsites.net/api/eventsource/rehydrate/RequestStat

e/prepare/all
Content-Type: application/json
Settings - {"throttleMs":5,"takePerThrottle":5}

2. Rehydrate all
   - This has to start after #1 completes.
   - This is also a background task and the endpoint will return an immediate `202 Accepted`.

POST request DEV
URL - https://eheu2dv1-as-ucxdecisionservice-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}

POST request INTG
URL - https://eheu2in1-as-ucxdecisionservice-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}

POST request PROD
URL - https://ehcuspd1-as-ucxdecisionservice-api.azurewebsites.net/api/eventsource/rehydrate/RequestState/all
Settings - {"throttleMs":5,"takePerThrottle":50,"whereRehydrateRequiredIsTrue":true}

Note: This background process may interrupt due to deployments and need to re-trigger once deployment complete. Prepare/Rehydration will resume where we left after interruption. By hitting same end points again, we may start new instance of the process parallel.